**Department of Electrical Engineering**

**Indian Institute of Technology, Kharagpur**

**Algorithms, AI and ML Laboratory (EE22202)**

**Spring, 2025-26**

# Report 5: Random Forests

**Name: Arijit Dey**

**Roll No: 24IE10001**

# Random Forests

## 1 Loading the Dataset

This section focuses on loading the dataset and performing an initial inspection of its attributes and target variable.

**Data Loading and Display**

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

cols = ['buying', 'maint', 'doors', "persons", "lug_boot", "safety", "accept"]

df = pd.read_csv("car.data", header=None, names=cols)

attributes = list(df.columns[:-1])
target = df.columns[-1]
X = df[attributes]
y = df[target]
```

**Result**

```
Attributes:
  buying  maint doors persons lug_boot safety
0  vhigh  vhigh     2       2    small    low
1  vhigh  vhigh     2       2    small    med
2  vhigh  vhigh     2       2    small   high
3  vhigh  vhigh     2       2      med    low
4  vhigh  vhigh     2       2      med    med

Target:
0    unacc
1    unacc
2    unacc
3    unacc
4    unacc
Name: accept, dtype: str
```

## 2 Model Definitions

This section defines the core classes for the ID3 Decision Tree and the RandomForestClassifier, which will be used for building and evaluating the random forest model.

## 3 ID3 Decision Tree Class

We will re-use the `ID3DecisionTree` from experiment 4 and extend it with two new functions `get_feature_depths()` and `_get_feature_depths_recursive()`. These helper functions are used by the `RandomForestClassifier` to get the obtain the minimum depth at which each feature is used for classification.

ID3DecisionTree Class Definition

```python
class ID3DecisionTree:
    # ... PREVIOUS CODE FROM EXPERIMENT 4

    def _get_feature_depths_recursive(self, node, current_depth, feature_depths):
        if not isinstance(node, tuple):
            return

        attr, _, subtrees = node

        if attr not in feature_depths or current_depth < feature_depths[attr]:
            feature_depths[attr] = current_depth

        for val in subtrees:
            self._get_feature_depths_recursive(subtrees[val], current_depth + 1,
            feature_depths)

    def get_feature_depths(self):
        feature_depths = {}
        if self.tree:
            self._get_feature_depths_recursive(self.tree, 1, feature_depths)
        return feature_depths
```

# 4 Random Forest Classifier Class

The `RandomForestClassifier` class implements an ensemble learning method that constructs a multitude of decision trees during training and outputs the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees.

## 4.1 Class Initialization

RandomForestClassifier Class Definition

```python
class RandomForestClassifier:
    def __init__(self, numTrees, featureBagSize, dataBagSize,
                 impurity_metric='entropy', max_depth=None):
        # -- SETUP CODE --
```

## 4.2 The `.fit()` Function

The `.fit()` function is the most important method in the `RandomForestClassifier` class. It takes the training dataset and trains the classifier based on those data samples.

1. We create the decision trees as specified by `numTrees` and randomly select features (without replacement) and sample rows (with replacement) for each tree.
2. We train the decision trees with the its corresponding data sample and than test it with the out-of-bag samples for that particular tree.
3. Lastly, we add the tree to out forest and use it for further classification.

Training loop

```python
    def fit(self, X, y):
        num_samples = len(X)
        all_features = list(X.columns)

        sample_size = int((self.dataBagSize / 100.0) * num_samples)
```

```python

        for i in range(self.numTrees):
            tree = ID3DecisionTree(
                impurity_metric=self.impurity_metric,
                max_depth=self.max_depth
            )

            selected_features = np.random.choice(
                all_features, size=self.featureBagSize, replace=False
            ).tolist()

            indices = np.random.choice(X.index, size=sample_size, replace=True)

            oob_indices = [idx for idx in X.index if idx not in indices]

            X_subset = X.loc[indices, selected_features]
            y_subset = y.loc[indices]

            tree.fit(X_subset, y_subset)

            self.forest.append(tree)
            self.feature_subsets.append(selected_features)
            self.oob_indices_per_tree.append(oob_indices)
            self.root_features.append(tree.root_feature)
            self.all_feature_depths.append(tree.get_feature_depths())

            if oob_indices:
                oob_X_subset = X.loc[oob_indices, selected_features]
                oob_y_preds_for_tree = []
                for _, oob_row in oob_X_subset.iterrows():
                    oob_y_preds_for_tree.append(tree.predict(oob_row))

                self.oob_predictions_per_tree.append(
                    pd.DataFrame({'y_true': y.loc[oob_indices], 'y_pred':
                    oob_y_preds_for_tree, 'tree_idx': i})
                )
```

### 4.3 Out-of-Bag Accuracy

The `compute_oob_accuracy()` method provides an unbiased estimate of the model's generalization error without needing a separate validation set. It works by:

1. We gather the OOB predictions made during the `fit()` process for samples that were not part of a tree's training bootstrap
2. For each unique sample index in the training set, it identifies all trees that did not see that sample during training.
3. A majority vote is taken among these specific unseen trees.
4. The OOB accuracy is the ratio of correctly predicted samples (using their aggregated OOB votes) to the total number of samples that were OOB for at least one tree.

**Out of Box accuracy**

```python
    def compute_oob_accuracy(self, X_original, y_original):
        if not self.oob_predictions_per_tree:
```

```
            return 0.0

        all_oob_preds_df = pd.concat(self.oob_predictions_per_tree)
        oob_sample_preds = {}
        for idx in y_original.index:
            sample_oob_preds = all_oob_preds_df[all_oob_preds_df['y_true'].index == idx]
            if not sample_oob_preds.empty:
                votes = Counter(sample_oob_preds['y_pred'])
                oob_sample_preds[idx] = votes.most_common(1)[0][0]

        if not oob_sample_preds:
            return 0.0

        oob_y_true = pd.Series({idx: y_original.loc[idx] for idx in oob_sample_preds.keys()})
        oob_y_pred = pd.Series(oob_sample_preds)

        return (oob_y_true == oob_y_pred).mean()
```

### 4.4 Feature Significance

The `get_weighted_feature_significance()` method determines the relative importance of each attribute across the entire ensemble using a depth-based weighting system:

- The significance $S$ contribution of a feature in a tree with feature bag size $N_b$ and minimum feature depth $d$ is:

$$S = \frac{N_b - d + 1}{N_b}$$

- These scores are summed across all trees in the forest to produce a final weighted significance score for each attribute.

```
    def get_root_feature_counts(self):
        root_feature_counts = Counter(self.root_features)
        return dict(root_feature_counts)

    def get_weighted_feature_significance(self, all_available_features):
        weighted_significance = {feature: 0.0 for feature in all_available_features}

        for tree_idx, feature_depths in enumerate(self.all_feature_depths):
            if tree_idx >= len(self.feature_subsets):
                continue

            current_feature_bag_size = len(self.feature_subsets[tree_idx])

            if current_feature_bag_size == 0:
                continue

            for feature in all_available_features:
                if feature in feature_depths:
                    depth_of_feature = feature_depths[feature]
                    weighted_significance[feature] += (current_feature_bag_size - depth_of_feature + 1) / current_feature_bag_size
```

```

        return weighted_significance
```

## 5 Model Training and Evaluation

This section outlines the process of training the RandomForestClassifier and evaluating its performance on training, validation, and test datasets. It also includes the computation of out-of-bag accuracy and analysis of feature significance.

**RandomForestClassifier Training and Evaluation Setup**

```
train_df, val_df, test_df = prepare_data(df, train_seg=80, val_seg=10, percent_noise=0)

num_attributes = len(train_df.columns) - 1
rf_model = RandomForestClassifier(
    numTrees=50,
    featureBagSize=4,
    dataBagSize=80,
)

X_train = train_df.drop(columns=[target])
y_train = train_df[target]

X_val =  val_df.drop(columns=[target])
y_val = val_df[target]

X_test = test_df.drop(columns=[target])
y_test = test_df[target]

rf_model.fit(X_train, y_train)

oob_accuracy = rf_model.compute_oob_accuracy(X_train, y_train)
tr_accr = rf_model.compute_accuracy(X_train, y_train)
v_accr = rf_model.compute_accuracy(X_val, y_val)
ts_accr = rf_model.compute_accuracy(X_test, y_test)
```

**Result**

```
--- CLassifier Performance ---
Average accuracy for Out-of-Bag data examples: 85.89%
Training accuracy:  93.05%
Validation accuracy: 87.79%
Testing accuracy: 85.06%

 --- Attribute Stats ---
Frequency of features as tree root
    - safety: 38 times
    - persons: 11 times
    - buying: 1 times

 Significance of attributes
 - safety:  40.2500
 - persons:  29.0000
```

```
- buying:  23.5000
- lug_boot:  15.0000
- maint:  12.7500
- doors:  11.5000
```

## 6 Comparison with Single Decision Tree

To understand the performance of the Random Forest ensemble, we compare it against a single ID3 Decision Tree trained on the full set of features.

**Single Decision Tree Evaluation**

```
dt_model = ID3DecisionTree()
dt_model.fit(X_train, y_train)

train_acc = dt_model.compute_accuracy(train_df) * 100
val_acc = dt_model.compute_accuracy(val_df) * 100
test_acc = dt_model.compute_accuracy(test_df) * 100
```

**Result**

```
Training accuracy: 100.00%
Validation accuracy: 96.51%
Testing accuracy: 93.10%
```

### 6.1 Summary Comparison

| Metric | Random Forest | Decision Tree |
|---|---|---|
| Training Accuracy | 93.05% | 100.00% |
| Validation Accuracy | 87.79% | 96.51% |
| Testing Accuracy | 85.06% | 93.10% |

## 7 Discussion: Why is Random Forest Accuracy Lower?

In this experiment, the Random Forest classifier achieved a testing accuracy of 85.06%, while the single Decision Tree reached 93.10%. This difference highlights several key aspects of ensemble learning versus individual tree growth:

1. With `featureBagSize=4` out of 6 total attributes, each tree in the forest is deprived of two potentially crucial features. In the Car Evaluation dataset, certain attributes like `safety` and `persons` carry immense predictive power. If a tree randomly excludes these, its individual accuracy plummets, and even the ensemble's majority vote cannot fully recover the lost precision.
2. The `ID3DecisionTree` implementation drops used features at each split. A tree limited to 4 features can only reach a maximum depth of 4, whereas the single decision tree, having access to all 6 features, can grow deeper (up to 6 levels) to finer-grained partitions of the data.
3. Random Forests are primarily designed to reduce variance and prevent overfitting on complex, noisy datasets. The `car.data` set is relatively simple and deterministic, allowing a single deep decision tree to model the rules perfectly without significant overfitting. In such cases, the "bias" introduced by bagging (both data and features) outweighs the "variance reduction" benefits.

4. Using only 80% of the data for training each tree further limits their ability to see the full variety of samples, especially in a small dataset where every instance might be important for defining the decision boundary.

## 8 Conclusion

- The single Decision Tree significantly outperformed the Random Forest on the Car Evaluation dataset (93.10% vs 85.06%).
- The performance gap is primarily due to the **restricted feature space** and **limited tree depth** in the forest ensemble, which prevented individual trees from capturing the full logic of the data.
- Feature importance analysis consistently identified `safety` and `persons` as the dominant predictors.
- This comparison underscores that Random Forests are not universally superior; for small, clean, and deterministic categorical datasets, a single well-grown decision tree can be more effective than an ensemble of constrained learners.